04. 구문 분석 단계 상세 분석

개요

Cairo 컴파일러의 구문 분석 단계는 소스 코드를 토큰으로 분해하고 이를 추상 구문 트리(AST)로 변환하는 과정입니다. 이 단계는 `cairo-lang-parser` crate에서 구현되며, 렉서(Lexer)와 파서(Parser) 두 개의 주요 컴포넌트로 구성됩니다.

cairo-lang-parser Crate 구조 분석

주요 모듈 구성

```
cairo-lang-parser/
├── src/
| ├── lib.rs # 메인 라이브러리 진입점
| ├── lexer.rs # 토큰화 담당 렉서
| ├── parser.rs # 구문 분석 담당 파서
| ├── recovery.rs # 에러 복구 메커니즘
| ├── diagnostic.rs # 진단 및 에러 메시지
| ├── operators.rs # 연산자 우선순위 처리
| ├── validation.rs # 토큰 유효성 검증
| ├── utils.rs # 유틸리티 함수들
| ├── printer.rs # AST 출력 기능
```

```
| └── colored_printer.rs # 컬러 출력 기능
```

핵심 의존성

- `cairo-lang-syntax` : AST 노드 정의와 구문 종류
- `cairo-lang-filesystem` : 파일 시스템과 텍스트 스팬 처리
- `cairo-lang-diagnostics` : 진단 메시지 시스템
- `salsa` : 쿼리 기반 컴파일러 인프라

Lexer의 토큰화 과정

Lexer 구조체

```
pub struct Lexer<'a> {
db: &'a dyn Database,
text: &'a str,
previous_position: TextOffset,
current_position: TextOffset,
done: bool,
}
```

토큰화 과정

1. 문자 단위 처리

렉서는 입력 텍스트를 문자 단위로 순회하며 다음과 같은 방식으로 토큰을 식별합니다:

```rust
fn match_terminal(&mut self) -> LexerTerminal<'a> {
let leading_trivia = self.match_trivia(true);
let kind = if let Some(current) = self.peek() {
match current {
'0'..='9' => self.take_token_literal_number(),
'\'' => self.take_token_short_string(),
'"' => self.take_token_string(),
'a'..='z' | 'A'..='Z' | '_' => self.take_token_identifier(),
// ... 기타 문자들
}
} else {
TokenKind::EndOfFile
};
}
```

2. 토큰 종류 분류

리터럴 토큰:

- `LiteralNumber` : 숫자 리터럴 (10진수, 16진수, 8진수, 2진수)
- `ShortString` : 단일 따옴표 문자열 ('hello')
- `String` : 이중 따옴표 문자열 ("hello")

키워드 토큰:

```rust
match self.peek_span_text() {
```

```
"as" => TokenKind::As,
"const" => TokenKind::Const,
"fn" => TokenKind::Function,
"struct" => TokenKind::Struct,
"enum" => TokenKind::Enum,
"let" => TokenKind::Let,
"match" => TokenKind::Match,
// ... 기타 키워드들
_ => TokenKind::Identifier,
}
```

연산자 및 구두점:

- 단일 문자: `+`, `-`, `*`, `/`, `=`, `<`, `>` 등
- 복합 문자: `==`, `!=`, `<=`, `>=`, `&&`, `||`, `->` 등

3. Trivia 처리

렉서는 공백, 주석, 개행 등의 trivia를 별도로 처리합니다:

```
fn match_trivia(&mut self, leading: bool) -> Vec<TriviumGreen<'a>> {
let mut res: Vec<TriviumGreen<'a>> = Vec::new();
while let Some(current) = self.peek() {
let trivium = match current {
' ' | '\r' | '\t' => self.match_trivium_whitespace(),
'\n' => self.match_trivium_newline(),
```

```
'/' if self.peek_nth(1) == Some('/') => self.match_trivium_single_line_comment
_ => break,
};
res.push(trivium);
}
res
}
```

특수 토큰 처리

숫자 리터럴

```
fn take_token_literal_number(&mut self) -> TokenKind {
// 16진수, 8진수, 2진수 처리
let special = if self.peek() == Some('0') {
self.take();
match self.peek() {
Some('x') => { /* 16진수 */ },
Some('o') => { /* 8진수 */ },
Some('b') => { /* 2진수 */ },
_ => false,
}
} else {
false
```

```
};

// 타입 접미사 처리 (_u32, _felt252 등)

if self.peek() == Some('_') {

self.take_while(|c| c.is_ascii_alphanumeric() || c == '_');

}

}
```

문자열 리터럴

```
fn take_token_string_helper(&mut self, delimiter: char) {

self.take(); // 시작 따옴표

let mut escaped = false;

while let Some(token) = self.peek() {

self.take();

match token {

_ if escaped => escaped = false,

'\\' => escaped = true,

_ if token == delimiter => break,

_ => {}

};

}

}
```

Parser의 구문 분석 알고리즘

Parser 구조체

```
pub struct Parser<'a, 'mt> {
db: &'a dyn Database,
file_id: FileId<'a>,
lexer: Lexer<'a>,
next_terminal: LexerTerminal<'a>,
next_next_terminal: Option<LexerTerminal<'a>>,
pending_trivia: Vec<TriviumGreen<'a>>,
offset: TextOffset,
current_width: TextWidth,
last_trivia_length: TextWidth,
diagnostics: &'mt mut DiagnosticsBuilder<'a, ParserDiagnostic<'a>>,
pending_skipped_token_diagnostics: Vec<PendingParserDiagnostic>,
macro_parsing_context: MacroParsingContext,
}
```

파싱 전략

1. 재귀 하강 파서 (Recursive Descent Parser)

Cairo 파서는 재귀 하강 방식을 사용하여 문법 규칙을 처리합니다:

```
// 함수 파싱 예제
```

```
fn parse_function(&mut self) -> TryParseResult<FunctionGreen> {
// 'fn' 키워드 확인
self.expect_terminal(SyntaxKind::TerminalFunction)?;
// 함수명 파싱
let name = self.parse_identifier()?;
// 제네릭 매개변수 파싱 (선택적)
let generic_params = self.parse_option_generic_params();
// 매개변수 목록 파싱
let params = self.parse_param_list()?;
// 반환 타입 파싱 (선택적)
let return_type = self.parse_option_return_type();
// 함수 본문 파싱
let body = self.parse_function_body()?;
}
```

2. 연산자 우선순위 파싱

표현식 파싱에서는 연산자 우선순위를 고려한 파싱을 수행합니다:

```
fn parse_expression_with_precedence(&mut self, min_precedence: usize) -> TryPa
let mut left = self.parse_primary_expression()?;
while let Some(op_precedence) = self.get_binary_operator_precedence() {
if op_precedence < min_precedence {
break;
```

```
}
let operator = self.take_terminal();
let right = self.parse_expression_with_precedence(op_precedence + 1)?;
left = self.create_binary_expression(left, operator, right);
}
Ok(left)
}
```

3. Try-Parse 패턴

파서는 실패할 수 있는 파싱 시도를 위해 try-parse 패턴을 사용합니다:

```
pub type TryParseResult<GreenElement> = Result<GreenElement, TryParseFailure>;

pub enum TryParseFailure {
SkipToken, // 토큰을 건너뛰어야 함
DoNothing, // 아무것도 하지 않음
}
```

AST 구조와 노드 타입

SyntaxKind 열거형

모든 구문 요소는 `SyntaxKind` 열거형으로 분류됩니다:

```rust
pub  enum SyntaxKind {
// 표현식 노드들
ExprPath,
ExprFunctionCall,
ExprStructCtorCall,
ExprBinary,
ExprUnary,
ExprMatch,
ExprIf,
ExprLoop,
ExprWhile,
ExprFor,
ExprBlock,
// 문장 노드들
StatementLet,
StatementExpr,
StatementReturn,
StatementBreak,
StatementContinue,
// 아이템 노드들
ItemFunction,
ItemStruct,
```

```
ItemEnum,
ItemTrait,
ItemImpl,
ItemModule,
ItemUse,
// 패턴 노드들
PatternIdentifier,
PatternStruct,
PatternTuple,
PatternEnum,
// 터미널 토큰들
TerminalIdentifier,
TerminalLiteralNumber,
TerminalString,
TerminalLBrace,
TerminalRBrace,
// ... 기타 터미널들
}
```

Green Node 구조

AST는 Green Node 방식으로 구현되어 메모리 효율성을 제공합니다:

```
pub struct GreenNode<'a> {
```

```
pub kind: SyntaxKind,
pub details: GreenNodeDetails<'a>,
}

pub enum GreenNodeDetails<'a> {
Token(TokenGreen<'a>),
Node { children: Vec<GreenId<'a>>, width: TextWidth },
}
```

AST 노드 예제

함수 선언 노드

```
pub struct FunctionDeclaration<'db> {
node: SyntaxNode<'db>,
}

impl<'db> FunctionDeclaration<'db> {
pub fn signature(&self, db: &'db dyn Database) -> FunctionSignature<'db> {
// 함수 시그니처 반환
}
pub fn body(&self, db: &'db dyn Database) -> ExprBlock<'db> {
// 함수 본문 반환
}
```

```
}
```

표현식 노드 계층

```
pub enum Expr<'db> {
Path(ExprPath<'db>),
FunctionCall(ExprFunctionCall<'db>),
StructCtorCall(ExprStructCtorCall<'db>),
Binary(ExprBinary<'db>),
Unary(ExprUnary<'db>),
Match(ExprMatch<'db>),
If(ExprIf<'db>),
Block(ExprBlock<'db>),
// ... 기타 표현식 타입들
}
```

에러 복구 메커니즘

복구 전략

1. 토큰 건너뛰기

예상되지 않은 토큰을 만났을 때 해당 토큰을 건너뛰고 파싱을 계속합니다:

```
fn skip_until_recovery_set(&mut self, recovery_set: &[SyntaxKind]) {
while !recovery_set.contains(&self.current_kind())
```

```
&& self.current_kind() != SyntaxKind::TerminalEndOfFile {
self.skip_token();
}
}
```

2. 복구 지점 정의

특정 구문 요소들을 복구 지점으로 사용합니다:

```
// 모듈 아이템 키워드들을 복구 지점으로 사용
macro_rules!  module_item_kw {
() => {
SyntaxKind::TerminalConst
| SyntaxKind::TerminalEnum
| SyntaxKind::TerminalExtern
| SyntaxKind::TerminalFunction
| SyntaxKind::TerminalImpl
| SyntaxKind::TerminalModule
| SyntaxKind::TerminalStruct
| SyntaxKind::TerminalTrait
| SyntaxKind::TerminalType
| SyntaxKind::TerminalUse
};
```

```
}
```

3. 진단 메시지 생성

에러 발생 시 사용자에게 유용한 진단 메시지를 제공합니다:

```
pub enum ParserDiagnosticKind {
SkippedElement { element_name: String },
MissingToken(SyntaxKind),
MissingExpression,
MissingPathSegment,
MissingTypeClause,
InvalidNumericLiteralValue,
UnterminatedString,
ReservedIdentifier { identifier: String },
// ... 기타 에러 타입들
}
```

에러 처리 예제

누락된 토큰 처리

```
fn expect_terminal(&mut self, expected_kind: SyntaxKind) -> Result<TokenGreen,
if self.current_kind() == expected_kind {
Ok(self.take_terminal())
} else {
```

```
// 누락된 토큰에 대한 진단 메시지 생성
self.create_and_report_missing_token(expected_kind);
// 빈 토큰으로 복구
Ok(self.create_missing_terminal(expected_kind))
}
}
```

표현식 파싱 에러 복구

```
fn parse_expression_or_recover(&mut self) -> ExprGreen {
match self.try_parse_expression() {
Ok(expr) => expr,
Err(TryParseFailure::SkipToken) => {
self.skip_token_and_report_error("Expected expression");
self.create_missing_expression()
}
Err(TryParseFailure::DoNothing) => {
self.create_missing_expression()
}
}
}
```

파싱 과정 예제

간단한 함수 파싱

입력 코드:

```
fn add(a: u32, b: u32) -> u32 {
a + b
}
```

파싱 과정:

1. **토큰화**: `fn`, `add`, `(`, `a`, `:`, `u32`, `,`, `b`, `:`, `u32`, `)`, `->`, `u32`, `{`, `a`, `+`, `b`, `}`
2. **구문 분석**:

- `ItemFunction` 노드 생성
- `FunctionSignature` 파싱 (이름, 매개변수, 반환 타입)
- `ExprBlock` 파싱 (함수 본문)
- `ExprBinary` 파싱 (`a + b` 표현식)

AST 구조 결과

```
ItemFunction
├── FunctionSignature
| ├── TerminalFunction ("fn")
| ├── TerminalIdentifier ("add")
| ├── ParamList
| | ├── Param (a: u32)
```

```
│ │ └── Param (b: u32)
│ └── ReturnTypeClause (-> u32)
└── ExprBlock
└── ExprBinary
├── ExprPath ("a")
├── TerminalPlus ("+")
└── ExprPath ("b")
```

성능 최적화

1. 지연 파싱 (Lazy Parsing)

필요한 경우에만 세부 AST 노드를 생성합니다.

2. 메모리 효율적인 Green Tree

불변 Green Node 구조를 사용하여 메모리 사용량을 최적화합니다.

3. 인크리멘털 파싱

Salsa 프레임워크를 활용하여 변경된 부분만 다시 파싱합니다.

결론

Cairo 컴파일러의 구문 분석 단계는 효율적인 렉서와 견고한 에러 복구 메커니즘을 갖춘 재귀 하강 파서로 구현되어 있습니다. Green Tree 구조를 사용한 AST는 메모리 효율성과 인크리멘털 컴파일을 지원하며, 포괄적인 진단 시스템을 통해 개발자에게 유용한 에러 메시지를 제공합니다.